Desarrollo de Interfaces (0488) 2º Curso DAM

Memoria Técnica: Control de Acceso y Protección de Rutas

Ionic Framework + Angular Standalone Components + Functional Guards (CanActivateFn)

Alumno: Fabio Rieker

Repositorio GitHub: https://github.com/FabioRieker/ActividadGuard

Despliegue en Vercel: https://actividad-guard-six.vercel.app

Credenciales de Prueba: `admin / 123` (Rol: 'admin') | `operador / 123` (Rol: 'user')

1. Fundamentación Teórica y Arquitectura Funcional

En las versiones contemporáneas de Angular (v16+ y superiores), la arquitectura de navegación abandonó el paradigma heredado basado en interfaces de clases (`CanActivate`) en favor de las **Functional Guards (`CanActivateFn`)**.

- **Inyección Directa en Contexto:** Se utiliza la función nativa `inject()` (ej. `inject(AuthService)` , `inject(Router)`) sin necesidad de constructores de clases ni boilerplate de inyección jerárquica.
- **Redirecciones Atómicas con `UrlTree` :** A diferencia del clásico retorno de un valor booleano falso (que congela o cancela sin redirigir), la guarda retorna un árbol de enrutamiento `UrlTree` mediante `router.createUrlTree(['/ruta'])` . Esto ejecuta una redirección limpia, instantánea y transaccional en un único ciclo del enrutador.
- **Persistencia de Sesión Sincronizada:** Para evitar pérdidas de sesión por reinicio del contexto de JavaScript o al abrir la URL copiada en nuevas pestañas, el estado reactivo basado en **Angular Signals** se sincroniza de forma dual con `sessionStorage` y `localStorage` .

2. Implementación de Código Fuente Clave

2.1. Factoría de Guards de Roles Granulares (`src/app/core/guards/role.guard.ts`)

Cumple con el ejercicio de la Sección 4: valida el rol del usuario autenticado. Si un usuario con rol `user` (operador) introduce directamente en el navegador `/admin` , la guarda intercepta la petición y redirige a `/user-dashboard` .

```
import { inject } from '@angular/core';
import { CanActivateFn, Router, UrlTree } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const roleGuard = (allowedRole: string): CanActivateFn => {
  return (route, state): boolean | UrlTree => {
    const authService = inject(AuthService);
    const router = inject(Router);

    // 1. Validar autenticación activa
    if (!authService.isAuthenticated()) {
      return router.createUrlTree(['/login'], {
        queryParams: { returnUrl: state.url }
      });
    }

    const currentRole = authService.currentUser()?.role;

    // 2. Si cuenta con el rol autorizado, permitir paso
    if (currentRole === allowedRole) {
      return true;
    }

    // 3. Denegar acceso a /admin a usuarios 'user' y redirigir a su panel
    if (currentRole === 'user') {
      return router.createUrlTree(['/user-dashboard']);
    }

    return router.createUrlTree(['/admin']);
  };
};
```

2.2. Guarda de Autenticación Funcional (`src/app/core/guards/auth.guard.ts`)

Verifica si el usuario está autenticado en memoria o almacenamiento local antes de permitir el acceso a rutas protegidas.

```
import { inject } from '@angular/core';
import { CanActivateFn, Router, UrlTree } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state): boolean | UrlTree => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // Verificación reactiva
  if (authService.isAuthenticated()) {
    return true;
  }

  // Redirección atómica hacia la pantalla de login preservando la URL original
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url }
  });
};
```

2.3. Servicio de Autenticación con Angular Signals y Persistencia (`src/app/core/services/auth.service.ts`)

Manejo de reactividad con Signals y sincronización de estado para persistir la sesión al refrescar y al copiar la URL.

```
import { Injectable, signal, computed } from '@angular/core';

export interface UserSession {
  username: string;
  role: 'admin' | 'user';
  isAuthenticated: boolean;
}

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private readonly STORAGE_KEY = 'DAM2_AUTH_SESSION';

  // Signal reactivo para el estado en memoria
  private _currentUser = signal<UserSession | null>(this.readSessionFromStorage());

  public currentUser = this._currentUser.asReadonly();
  public isAuthenticated = computed(() => this._currentUser()?.isAuthenticated ?? false);

  public login(user: string, pass: string): boolean {
    let role: 'admin' | 'user' | null = null;
    if (user === 'admin' && pass === '123') role = 'admin';
    else if (user === 'operador' && pass === '123') role = 'user';

    if (role) {
      const sessionData: UserSession = { username: user, role, isAuthenticated: true };
      this._currentUser.set(sessionData);

      // Persistencia en Web Storage API
      const serialized = JSON.stringify(sessionData);
      sessionStorage.setItem(this.STORAGE_KEY, serialized);
      localStorage.setItem(this.STORAGE_KEY, serialized);
      return true;
    }
    return false;
  }

  public logout(): void {
    sessionStorage.removeItem(this.STORAGE_KEY);
    localStorage.removeItem(this.STORAGE_KEY);
    this._currentUser.set(null);
  }

  private readSessionFromStorage(): UserSession | null {
    const rawData = sessionStorage.getItem(this.STORAGE_KEY) || localStorage.getItem(this.STORAGE_KEY
    if (!rawData) return null;
    try {
      const session = JSON.parse(rawData) as UserSession;
      sessionStorage.setItem(this.STORAGE_KEY, rawData);
      localStorage.setItem(this.STORAGE_KEY, rawData);
      return session;
    } catch {
      this.logout();
      return null;
    }
  }
}
```

2.4. Enrutamiento Modular y Lazy Loading (`src/app/app.routes.ts`)

```
import { Routes } from '@angular/router';
import { authGuard } from './core/guards/auth.guard';
import { roleGuard } from './core/guards/role.guard';

export const routes: Routes = [
  {
    path: '',
    loadComponent: () => import('./pages/home/home.page').then(m => m.HomePage),
  },
  {
    path: 'login',
    loadComponent: () => import('./pages/login/login.page').then(m => m.LoginPage),
  },
  {
    path: 'admin',
    canActivate: [authGuard, roleGuard('admin')],
    loadComponent: () => import('./pages/admin/admin.page').then(m => m.AdminPage),
  },
  {
    path: 'user-dashboard',
    canActivate: [authGuard, roleGuard('user')],
    loadComponent: () => import('./pages/user-dashboard/user-dashboard.page').then(m => m.UserDashboa
  },
  {
    path: '**',
    redirectTo: '',
  },
];
```

3. Metodología Git y Estrategia de Ramas (Git Flow)

El repositorio sigue la estrategia de ramas y la convención de commits solicitada:

- `main` **:** Versiones cerradas, estables y listas para la evaluación del tribunal.
- `develop` **:** Rama base de integración diaria del equipo de trabajo.
- `tarea/control-acceso-guards` **:** Rama temporal de desarrollo de la funcionalidad.
- **Conventional Commits:** Commits con prefijos `feat:` (nuevas funcionalidades), `fix:` (correcciones) y `docs:` (documentación).

4. Demostración y Verificación Técnica

Prueba / Escenario	Procedimiento	Resultado Esperado	Estado
Página de Inicio	Cargar `/`	Muestra "Hola Mundo" y botón al login	✔ **Superado**
Login Admin	Pulsar botón rápido "Iniciar como Admin"	Toast de bienvenida y navegación a `/admin`	✔ **Superado**
Login Operador	Pulsar botón rápido "Iniciar como Operador"	Toast de bienvenida y navegación a `/user-dashboard`	✔ **Superado**
Ataque a /admin	Escribir `/admin` con rol 'user'	`roleGuard` intercepta y redirige a `/user-dashboard`	✔ **Superado**
Persistencia URL	Copiar URL a otra pestaña	Sesión conservada activamente mediante Web Storage	✔ **Superado**
Cierre de Sesión	Pulsar "Cerrar Sesión"	Toast de despedida, borrado de storages y vuelta a `/login`	✔ **Superado**